

Security Report Revised

Generated from	Security_report_revised.md
Review date	2026-06-12

Date: 2026-06-12

Scope: Django backend, Vite/React frontend, project configuration, and deployment guidance visible in this repository.

Review mode: Security review only. No application code changes were performed.

Executive Summary

The previous high-risk findings were substantially remediated: Django is now on the 5.2 LTS range, production settings can fail closed when DJANGO_ENV=production, OpenAI API keys are encrypted at rest, uploads have size and DOCX archive checks, checklist generation is queued, private storage is used for uploaded/generated files, CORS is exact-origin, public API IDs are UUIDs, and frontend dependency audit currently reports 0 vulnerabilities.

The revised review found no critical code-level vulnerability in the tracked application source. The remaining risks are mostly conditional production and operational issues: the system still uses SQLite as its only configured database, local defaults are unsafe if accidentally deployed, static frontend security headers depend on external hosting configuration, throttling is only as strong as the database/proxy environment, and AI/data-retention controls remain partly procedural.

Current Database Identified

The system currently uses SQLite.

Location: backend/volunteer_portal/settings.py:94-98

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

The expected local database file is backend/db.sqlite3. It is ignored by git (.gitignore:5-6) and was not tracked by git ls-files, but it exists in the local workspace as an ignored file.

Risk Assessment Matrix

Risk score = Likelihood x Impact.

Score	Severity	Meaning
20-25	Critical	Likely exploitation with severe compromise or disclosure
12-19	High	Plausible exploitation with high operational or security impact
6-11	Medium	Meaningful risk requiring remediation, often conditional
1-5	Low	Defense-in-depth, hardening, or limited-impact issue

Likelihood	Description
---	---

5	Expected or trivially reachable
4	Likely in realistic deployment or normal use
3	Possible with some access or configuration condition
2	Unlikely or requires elevated/preconditioned access
1	Rare or mostly theoretical

Impact	Description
5	Full app compromise, credential/session compromise, or major sensitive data disclosure
4	High-value secret exposure, broad data exposure, or significant service disruption/cost
3	Limited data exposure, targeted disruption, or important defense bypass
2	Information leakage, audit gaps, or narrow abuse
1	Minor hardening gap

Ranked Findings

REV-01: SQLite Is the Only Configured Database

Severity: High

Likelihood: 3

Impact: 4

Risk score: 12

Rule IDs: Django production configuration, data protection, availability hardening

Location: backend/volunteer_portal/settings.py:94-98

Evidence:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

Impact: If used in production, a single SQLite file becomes a high-value local artifact containing users, sessions, encrypted OpenAI settings, resource metadata, throttle records, and checklist job metadata. SQLite also limits production-grade access control, connection security, backups, concurrent writes, and separation of application and data privileges.

Fix summary: Add an environment-driven production database configuration, preferably PostgreSQL or another managed production database. Load credentials from a secret manager or environment, require TLS where supported, use a least-privilege database user, configure backups, and keep SQLite only for local development/tests.

Mitigation: If SQLite must be used temporarily, keep the database file outside web roots, restrict filesystem permissions, encrypt server volumes/backups, and document recovery/backup handling. This should be temporary only for a multi-user authenticated portal.

False positive notes: SQLite is acceptable for local development. This finding is production-impacting because no alternate production database configuration is visible in code.

REV-02: Local Defaults Remain Unsafe If Accidentally Deployed

Severity: High

Likelihood: 3

Impact: 4

Risk score: 12

Rule IDs: DJANGO-DEPLOY-002, DJANGO-CONFIG-001, DJANGO-HTTPS-001

Location: backend/volunteer_portal/settings.py:34-49, backend/volunteer_portal/settings.py:134-139, frontend/src/main.jsx:19

Evidence:

```
DJANGO_ENV = os.environ.get("DJANGO_ENV", "local").strip().lower()
IS_PRODUCTION = DJANGO_ENV == "production"
SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY")
...
DEBUG = env_bool("DJANGO_DEBUG", not IS_PRODUCTION)
```

```
CSRF_COOKIE_SECURE = env_bool("DJANGO_CSRF_COOKIE_SECURE", IS_PRODUCTION)
SESSION_COOKIE_SECURE = env_bool("DJANGO_SESSION_COOKIE_SECURE", IS_PRODUCTION)
SECURE_SSL_REDIRECT = env_bool("DJANGO_SECURE_SSL_REDIRECT", IS_PRODUCTION)
```

```
const API_BASE = (import.meta.env.VITE_API_BASE_URL || `http://${window.location.hostname}:8000`).replace(/\/$/, "");
```

Verification evidence: Running `uv run python backend/manage.py check --deploy` with default local settings reported 5 deployment warnings: `DEBUG=True`, missing `SECURE_SSL_REDIRECT`, missing secure session cookie, missing secure CSRF cookie, and unset HSTS. Running the same check with production-like environment variables reported no issues.

Impact: A deployment that forgets `DJANGO_ENV=production` can expose Django debug pages, use a known local-only secret key, and send cookie-authenticated traffic through HTTP defaults. The frontend also falls back to `http://<host>:8000` if the production build omits `VITE_API_BASE_URL`.

Fix summary: Make production deployment fail closed unless an explicit environment is set. Consider separate settings modules (local and production), a deployment endpoint that refuses local mode, and a frontend production build check that requires `VITE_API_BASE_URL` or uses a reviewed same-origin API path.

Mitigation: Keep `manage.py check --deploy` in deployment verification, ensure the production environment variables from `CONFIGURE.md` are mandatory in hosting configuration, and monitor response headers after deployment.

False positive notes: The production path itself can pass Django's deploy check when configured correctly. The risk is accidental deployment of local defaults.

REV-03: Static React Frontend Security Headers Are Not Enforced by Repository Code

Severity: Medium

Likelihood: 3

Impact: 3

Risk score: 9

Rule IDs: REACT-CSP-001, JS-CSP-001, clickjacking defense

Location: frontend/index.html:3-10, backend/resources/middleware.py:36-38, CONFIGURE.md:165-179

Evidence:

Volunteer Resource Portal

```
if settings.CSP_ENABLED:
    header_name = "Content-Security-Policy-Report-Only" if settings.CSP_REPORT_ONLY else "Content-Security-Policy"
    response.setdefault(header_name, settings.CSP_POLICY)
```

CONFIGURE.md instructs operators to configure CSP at the CDN/static server for the React app, but no static host config is present in this repo.

Impact: The Django backend can emit CSP for backend responses, but the browser-delivered React app may have no CSP, frame-ancestors, or other static-host security headers unless the deployment platform is configured correctly. If a future DOM XSS bug is introduced, missing CSP reduces browser-level containment.

Fix summary: Add deployment-managed headers for the static frontend host, including CSP, frame-ancestors or equivalent clickjacking protection, X-Content-Type-Options, and a suitable referrer policy. If the hosting platform cannot set headers, add a very early CSP meta tag as a partial fallback, noting that meta CSP cannot enforce all directives.

Mitigation: Verify headers against the live frontend with curl -I or browser security tooling before production release. Continue avoiding dangerouslySetInnerHTML, innerHTML, eval, and third-party scripts.

False positive notes: The security headers may be configured outside this repository. They are not visible here, so this must be verified at runtime.

REV-04: Login Throttling Depends on SQLite and REMOTE_ADDR

Severity: Medium

Likelihood: 3

Impact: 3

Risk score: 9

Rule IDs: DJANGO-AUTH-001, DJANGO-ADMIN-001, rate-limit reliability

Location: backend/resources/throttling.py:14-35, backend/resources/views.py:58-60, backend/resources/middleware.py:47-55, backend/volunteer_portal/settings.py:94-98

Evidence:

```
def client_ip(request):
    return request.META.get("REMOTE_ADDR", "")
```

```
with transaction.atomic():
    record, _created = ThrottleRecord.objects.select_for_update().get_or_create(...)
```

```
key_hash = throttle_key(client_ip(request), username)
```

Impact: The current throttle implementation is useful, but its accuracy depends on a production database and correct proxy configuration. SQLite does not provide the same row-level locking behavior expected by select_for_update, and REMOTE_ADDR may be a proxy IP rather than the client IP. Misconfiguration can cause either under-throttling or accidental lockouts.

Fix summary: Move production to a database with proper row locking, or use a battle-tested limiter such as django-axes or a Redis-backed rate limiter. Configure trusted proxy headers carefully, and consider multiple throttle dimensions: IP, username,

IP plus username, and admin-specific controls. Add MFA/SSO or network restrictions for Django admin.

Mitigation: Keep strong passwords and admin access restrictions. Monitor failed login attempts and rate-limit at the reverse proxy/WAF as a second layer.

False positive notes: This is not a total absence of throttling. The concern is reliability under realistic deployment and proxy conditions.

REV-05: Resource Access Is All-or-Nothing for Authenticated Users

Severity: Medium

Likelihood: 3

Impact: 3

Risk score: 9

Rule IDs: authorization and least privilege

Location: backend/resources/views.py:86-96, backend/resources/views.py:117-128, backend/resources/models.py:32-49

Evidence:

```
@api_login_required
@require_GET
def resource_list(request):
    resources = Resource.objects.filter(is_active=True)
```

```
@api_login_required
@require_GET
def resource_download(request, resource_id):
    resource = Resource.objects.get(public_id=resource_id, is_active=True)
```

```
class Resource(models.Model):
    title = models.CharField(max_length=180)
    description = models.TextField(blank=True)
    category = models.CharField(max_length=20, choices=Category.choices)
```

Impact: Any authenticated user can list and download every active resource. If future resources contain cohort-specific, staff-only, location-sensitive, or partner-confidential material, a normal volunteer account would be enough to access it.

Fix summary: Add per-resource authorization. Common options are resource visibility levels, groups/roles, per-category permissions, or explicit allowed user/group relations. Enforce the same access check in both list and download endpoints.

Mitigation: Treat all active resources as visible to every authenticated user until finer-grained authorization exists. Keep sensitive files out of this resource library.

False positive notes: This may match current product requirements. It becomes a security issue if resources vary by audience or sensitivity.

REV-06: AI Data Transfer Controls Are Still Best-Effort

Severity: Medium

Likelihood: 3

Impact: 3

Risk score: 9

Rule IDs: AI data handling, data minimization, prompt boundary

Location: backend/resources/checklist_generator.py:70-75, backend/resources/checklist_generator.py:115-124, CONFIGURE.md:181-193

Evidence:

```
REDACTION_PATTERNS = [
    (... , "[redacted-email]"),
    (... , "[redacted-phone]"),
    (... , "[redacted-token]"),
    (... , r"\1=[redacted-secret]"),
]
```

```
safe_concept_note = redact_sensitive_text(concept_note_text)
...
{"role": "user", "content": USER_PROMPT_TEMPLATE.format(concept_note=safe_concept_note[:80000])},
```

Impact: The code redacts obvious emails, phone-like values, and common secret formats, but concept notes may still include names, school locations, schedules, internal procedures, participant details, or other confidential information. Up to 80,000 characters are sent to the OpenAI API for processing.

Fix summary: Add explicit in-app notice and consent before AI processing. Classify concept notes before submission, strengthen data minimization, allow admins to disable AI generation, and document provider retention, project isolation, and data processing terms. Consider stricter redaction/DLP or manual review for sensitive events.

Mitigation: Limit `can_generate_checklist` to trained users, keep provider spend and usage monitoring enabled, and continue reviewing generated PDFs before sharing.

False positive notes: The prompt includes an instruction boundary and output validation. This finding is about privacy/governance and residual data exposure, not direct code execution.

REV-07: Expired Checklist File Cleanup Requires External Scheduling

Severity: Medium

Likelihood: 4

Impact: 2

Risk score: 8

Rule IDs: data retention, private file lifecycle

Location: backend/resources/cleanup.py:7-17, backend/resources/tasks.py:48-50, CONFIGURE.md:139-144

Evidence:

```
def delete_expired_checklist_jobs(batch_size=None):
    jobs = list(ChecklistJob.objects.filter(expires_at__lte=timezone.now())[ :batch_size])
    for job in jobs:
        for field in (job.concept_note, job.generated_pdf):
            if field:
                field.delete(save=False)
        job.delete()
```

```
@shared_task(ignore_result=True)
def cleanup_expired_checklist_jobs():
    delete_expired_checklist_jobs()
```

CONFIGURE.md tells operators to schedule cleanup with Celery or cron, but no Celery beat schedule, cron manifest, or deployment scheduler is present in this repo.

Impact: If operators forget to schedule cleanup, expired concept notes and generated PDFs remain in private storage. They are hidden from normal API lists after expiry, but still exist on disk/backups and may remain subject to disclosure via admin, filesystem, or backup access.

Fix summary: Add a deployment-owned schedule for cleanup_expired_checklist_jobs or the cleanup_checklist_jobs management command. Prefer a checked-in Celery beat schedule or platform scheduler configuration for production.

Mitigation: Monitor private media size and expired job counts. Run cleanup manually during operations until scheduling is confirmed.

False positive notes: The deletion implementation exists and appears reasonable. The residual risk is operational scheduling.

REV-08: Celery/Redis Broker Hardening Is Not Enforced

Severity: Medium

Likelihood: 2

Impact: 3

Risk score: 6

Rule IDs: queue security, deployment secret management

Location: backend/volunteer_portal/settings.py:151-153, CONFIGURE.md:50-51

Evidence:

```
CELERY_BROKER_URL = os.environ.get("CELERY_BROKER_URL", "redis://localhost:6379/0")
CELERY_RESULT_BACKEND = os.environ.get("CELERY_RESULT_BACKEND", CELERY_BROKER_URL)
CELERY_TASK_ALWAYS_EAGER = env_bool("CELERY_TASK_ALWAYS_EAGER", False)
```

```
CELERY_BROKER_URL=redis://redis.example.org:6379/0
CELERY_RESULT_BACKEND=redis://redis.example.org:6379/0
```

Impact: If Redis is exposed or unauthenticated in production, attackers with broker access may enqueue tasks or interfere with checklist processing. Current tasks mostly use database IDs rather than sending uploaded file contents through the broker, which limits data exposure, but task execution and availability remain sensitive.

Fix summary: Require production broker URLs explicitly, use authenticated Redis or a managed broker, restrict network access, enable TLS where supported, and avoid exposing Redis publicly. Use separate broker databases/credentials per environment.

Mitigation: Keep Redis bound to private networks only and monitor unexpected tasks or broker connections.

False positive notes: The default is appropriate for local development. The finding is conditional on production broker configuration.

REV-09: Uploaded Resource Files Are Not Malware-Scanned or Content-Disarmed

Severity: Medium

Likelihood: 2

Impact: 3

Risk score: 6

Rule IDs: file upload safety, untrusted content distribution

Location: backend/resources/models.py:14-29, backend/resources/models.py:41-46, backend/resources/views.py:117-128

Evidence:

```
if extension == ".pdf":
    if uploaded_file.read(5) != b"%PDF-":
        raise ValidationError("Upload a valid PDF file.")
elif extension == ".docx":
    validate_docx_archive(uploaded_file)
```

```
pdf_file = models.FileField(
    "resource file",
    storage=PrivateMediaStorage(),
    upload_to="resources/files/",
    validators=[FileExtensionValidator(["pdf", "docx"]), validate_resource_file],
)
```

Impact: Staff/admin users can upload PDF/DOCX resources that are later distributed to authenticated volunteers. The app validates extension and basic structure, but it does not scan for malware, embedded active content, suspicious links, or document exploit payloads.

Fix summary: Add antivirus scanning and/or content disarm and reconstruction for uploaded resources before activation. Consider an approval workflow where files are inactive until scanning passes. Restrict upload permission to trusted staff.

Mitigation: Continue forcing resource downloads as attachments, educate admins to upload trusted documents only, and restrict admin access.

False positive notes: This is mainly a compromised-admin or malicious-document risk. The existing validation reduces malformed file risk but does not replace malware scanning.

REV-10: Resource Search Input Is Not Length-Bounded

Severity: Low

Likelihood: 2

Impact: 2

Risk score: 4

Rule IDs: input validation, availability hardening

Location: backend/resources/views.py:89-96

Evidence:

```
search = request.GET.get("search", "").strip()
...
if search:
    resources = resources.filter(Q(title__icontains=search) | Q(description__icontains=search))
```

Impact: Authenticated users can submit very long search strings that cause unnecessary database work and memory use. With SQLite and unindexed case-insensitive contains queries, this is a low-grade availability concern.

Fix summary: Cap search length, reject or truncate excessive values, add pagination, and consider indexed/search-specific fields if the resource library grows.

Mitigation: Keep resource-list access authenticated and monitor slow queries/request sizes.

False positive notes: The current dataset may be small, so this is a hardening issue rather than an immediate vulnerability.

REV-11: Security Audit Logging Is Limited

Severity: Low

Likelihood: 2

Impact: 2

Risk score: 4

Rule IDs: monitoring, incident response, accountability

Location: backend/resources/views.py:48-69, backend/resources/views.py:152-176, backend/resources/admin.py:7-69

Evidence:

```
user = authenticate(request, username=username, password=password)
...
login(request, user)
```

```
job = ChecklistJob.objects.create(...)
generate_checklist_job.delay(job.id)
```

No structured application logs or audit records are visible for successful/failed API logins, checklist generation, OpenAI setting changes, resource uploads, or resource downloads.

Impact: If an account is compromised or sensitive files are misused, operators may have limited evidence to investigate who accessed or generated what, when, and from where.

Fix summary: Add structured security logging for authentication events, admin configuration changes, resource upload/download, checklist job creation, and cleanup. Send logs to a protected central log system with retention and alerting.

Mitigation: Use Django admin history as a partial signal for admin changes and retain web server access logs.

False positive notes: Infrastructure logging may exist outside the repo. It was not visible in application code.

REV-12: Security Checks Are Not Automated in Repository CI

Severity: Low

Likelihood: 2

Impact: 2

Risk score: 4

Rule IDs: dependency governance, secure release process

Location: repository root; no .github/workflows, Dockerfile, Procfile, or deployment manifest was found in the reviewed tree.

Evidence:

Manual verification in this review found:

```
npm audit --json: 0 total vulnerabilities
production-like manage.py check --deploy: no issues
```

No checked-in CI workflow was visible to run these checks continuously.

Impact: Security regressions such as accidental DEBUG=True, missing production env, vulnerable dependencies, migration drift, or frontend build failures may not be caught before deployment.

Fix summary: Add CI that runs backend tests, manage.py check --deploy with production-like settings, migration checks, frontend build, npm audit, and a Python dependency audit against uv.lock.

Mitigation: Keep running the documented verification steps manually before release until CI exists.

False positive notes: CI may exist outside this repository or in a hosting platform. It is not visible in the codebase reviewed.

Previous Report Status

```
| Previous ID | Current status |
| --- | --- |
| SEC-01 Unsafe production Django settings | Mostly resolved for `DJANGO_ENV=production`; residual accidental-local-deploy risk tracked as REV-01 |
| SEC-02 Unsupported Django version | Resolved; `pyproject.toml:8` uses `Django>=5.2,<6.0` |
| SEC-03 Plaintext OpenAI API key | Resolved in code; encrypted field and masked admin form are present |
| SEC-04 Media persistence/direct access | Mostly resolved with private storage and cleanup implementation; cleanup scheduling tracked as REV-02 |
| SEC-05 DOCX upload limits | Resolved; upload size and DOCX archive limits are present |
| SEC-06 Synchronous/costly checklist generation | Resolved with Celery queue and throttling; broker/data governance tracked as REV-03 |
| SEC-07 Missing login throttling | Improved; residual proxy/SQLite reliability tracked as REV-04 |
| SEC-08 Missing CSP | Partially resolved for Django responses; frontend host enforcement tracked as REV-03 |
| SEC-09 Dev-only CORS | Resolved; exact configured origin CORS is present |
| SEC-10 Sequential public IDs | Resolved; resource and checklist public API identifiers are UUIDs |
| SEC-11 AI prompt/data handling | Improved with redaction/output bounds; residual governance tracked as REV-06 |
| SEC-12 Backend dependency locking | Resolved with `uv.lock`; CI/audit automation tracked as REV-12 |
```

Positive Observations and Non-Findings

- CSRF middleware is enabled and state-changing API views use @csrf_protect.
- React POST requests include X-CSRFToken, and cookie credentials are explicit.
- No csrf_exempt, raw SQL string formatting, shell execution, unsafe pickle/yaml deserialization, eval, new Function, document.write, innerHTML, or dangerouslySetInnerHTML use was found in tracked application source.
- API resource and checklist job URLs now use UUIDs.
- Checklist job preview/download enforces both user ownership and unexpired job state.
- Private uploads use PrivateMediaStorage with base_url=None.
- Resource downloads force attachment disposition.
- Checklist PDF rendering escapes model-generated text before passing it to ReportLab paragraphs.
- npm audit --json reported 0 vulnerabilities for the frontend dependency tree.

- A production-like Django deploy check reported no issues when required production environment variables were supplied.

Recommended Remediation Order

1. Add production database configuration and move production away from SQLite.
2. Make accidental local-mode deployment harder through explicit settings modules or deployment endpoint checks.
3. Add checked-in or platform-managed static frontend security headers and verify them at runtime.
4. Strengthen throttling with a production-grade database/Redis limiter and trusted proxy configuration.
5. Decide whether resources need per-group/per-role authorization and implement it before storing sensitive resources.
6. Add AI data-handling UI notice, consent, and stronger minimization controls.
7. Add a checked-in scheduled cleanup mechanism for expired checklist jobs/files.
8. Harden Celery/Redis deployment credentials and network exposure.
9. Add malware scanning or approval workflow for uploaded resource files.
10. Add CI/security automation and structured audit logging.

Verification Performed

- Used the local security-best-practices skill and reviewed the Django, general frontend JavaScript, and React security references.
- Reviewed current backend settings, middleware, models, views, upload validation, storage, tasks, cleanup, admin, frontend app code, package manifests, and configuration guide.
- Searched for common risky patterns: CSRF exemptions, broad CORS, raw SQL, shell execution, unsafe deserialization, DOM XSS sinks, local/session storage token use, dynamic redirects, secrets, deployment commands, and untracked sensitive artifacts.
- Ran `npm audit --json` in frontend; result: 0 vulnerabilities.
- Ran `uv run python backend/manage.py check --deploy` with default local settings; result: 5 warnings expected from local defaults.
- Ran `manage.py check --deploy` with production-like environment variables; result: no issues.
- Confirmed `backend/db.sqlite3`, `backend/media/`, `backend/private_media/`, and `frontend/node_modules/` are ignored by git.

References

- Local security guidance:
- `/Users/natpatchara/.codex/skills/security-best-practices/references/python-django-web-server-security.md`
- `/Users/natpatchara/.codex/skills/security-best-practices/references/javascript-general-web-frontend-security.md`
- `/Users/natpatchara/.codex/skills/security-best-practices/references/javascript-typescript-react-web-frontend-security.md`
- Existing prior report: `security_best_practices_report.md`
- Deployment guide reviewed: `CONFIGURE.md`